Company Knowledge Assistant

A step-by-step RAG project plan | 38 tasks across 16 phases

Goal: upload company PDFs, retrieve relevant passages, answer with page citations, compare retrieval strategies, evaluate quality, and deploy a working application.

Suggested stack: Python + FastAPI; Next.js + TypeScript; PostgreSQL + pgvector; one embedding model and one LLM; Docker Compose.

00 Project setup

1. Create the project

Create backend/, frontend/, documents/, tests/, .env.example, and README.md. Create a Python virtual environment in backend/; install FastAPI, Uvicorn, and python-dotenv.

2. Create a basic FastAPI server

Implement GET /health returning {"status":"ok"}. Explain the request-response cycle before moving on.

01 Understand documents

3. Collect test documents

Prepare 3-5 known PDFs: employee handbook, leave policy, remote work policy, product guide, and FAQ. Use fictional data.

4. Extract PDF text

Implement document_loader.py with load_pdf(path) returning page number and text for each page. Inspect the extracted text and preserve page metadata.

02 Chunking

5. Build a chunker

Write chunker.py yourself. Start near 500 characters with 100 characters overlap; return text, page, and chunk_index for each chunk.

6. Experiment with chunk sizes

Compare sizes 200, 500, 1000, and 1500; compare overlap 0, 50, 100, and 200. Check whether a chunk independently contains enough answer context.

03 Embeddings

7. Generate an embedding

Choose one model or API. Embed a leave-policy sentence; print vector dimension and the first 10 values.

8. Compare semantic similarity

Embed the leave sentence, a vacation question, and an unrelated Python sentence. Compute cosine similarity and explain the ranking.

Checkpoint: Explain how chunking and embeddings affect retrieval before adding the database.

04 PostgreSQL + pgvector

9. Create the database

Create company_rag and enable CREATE EXTENSION vector. Confirm the extension is available in your PostgreSQL installation.

10. Design the schema

Create documents and document_chunks tables. Store document ID, content, embedding, page number, chunk index, filename or source reference, and timestamps.

11. Insert embeddings

Run PDF -> parse -> chunk -> embed -> database. Manually inspect stored rows and their metadata.

05 First retrieval system

12. Embed each question

Convert a user question into a vector with the same embedding model used for the documents.

13. Run vector search

Retrieve the top 5 chunks with pgvector and print their content and source. Test at least 20 questions; record expected document, retrieved document, and pass/fail. Keep the LLM out of this stage.

Checkpoint: Retrieval works on your test questions before an LLM is introduced.

06 First complete RAG

14. Build context

Format the top retrieved chunks into a bounded context with stable source IDs.

15. Write the RAG prompt

Tell the model to answer only from supplied context and say when evidence is insufficient. Include the question and source-labeled chunks.

16. Generate an answer

Send the prompt to one LLM. Confirm question -> retrieve -> context -> answer works end to end.

07 Sources and citations

17. Preserve source metadata

Carry document ID, filename, page, and chunk index through ingestion and retrieval.

18. Return citations

Return answer plus source entries containing filename, page, and chunk ID. Check each cited passage actually supports its claim.

08 Build the API

19. Upload endpoint

POST /documents: validate PDF, save, parse, chunk, embed, and store. Return document status or a clear error.

20. Document endpoints

GET /documents and DELETE /documents/{id}. Ensure deleting a document removes its chunks.

21. Chat endpoint

POST /chat accepts a question and returns answer and sources. Validate empty questions and handle retrieval/model errors.

Milestone: A complete RAG API with grounded answers and sources.

09 Next.js interface

22. Document management page

Build upload, list, processing state, and delete controls.

23. Chat interface

Show the conversation and clickable filename/page citations leading to the supporting passage. Keep the UI simple.

10 Improve retrieval

24. Show similarity scores

Expose retrieval ranks and scores in debug output; interpret scores for your chosen metric instead of imposing a universal cutoff.

25. Add metadata filters

Allow retrieval restricted to selected documents.

26. Add lexical search

Implement keyword/full-text search; understand BM25 as a ranking concept and compare it with vectors on the same questions.

27. Combine hybrid results

Merge vector and lexical candidates with a documented rank-fusion method. Re-run your test questions.

11 Reranking

28. Retrieve more candidates

Expand retrieval to about 15-20 candidate chunks.

29. Add a reranker

Rerank those candidates and pass the best 3-5 to the model. Compare results and latency with your earlier pipeline.

12 Query rewriting

30. Resolve follow-up questions

Rewrite an ambiguous follow-up such as "What about interns?" into a standalone search query using relevant conversation context. Keep the original user wording for answering.

13 Evaluation

31. Create a question set

Write at least 30 questions with expected answer and source. Include unanswerable questions and paraphrases.

32. Evaluate retrieval

Measure whether the expected source appears in top K, and inspect misses. Calculate hit rate and recall-oriented metrics.

33. Evaluate answers

Judge relevance, factual support, citation accuracy, and appropriate refusal. Start with manual checks, then try DeepEval or RAGAS.

34. Compare systems

Run the same dataset through vector-only, hybrid, and hybrid-plus-reranker variants. Report quality and latency; retain changes that help.

Milestone: An evidence-based comparison of retrieval and answer quality.

14 RAG debug dashboard

35. Build /debug

Show original and rewritten queries, ranked retrieved chunks, reranked order, chosen context, token count, latency, answer, and citations. Restrict debug details to authorized users in a deployed app.

15 Production basics

36. Harden the app

Add environment configuration, input validation, consistent errors, logs, loading states, streaming where useful, and file size/type restrictions.

37. Containerize

Create Docker images for Next.js and FastAPI and run them with PostgreSQL + pgvector in Docker Compose.

38. Deploy and document

Deploy the stack; verify upload, question answering, citations, deletion, and errors. Add setup, architecture, evaluation results, and deployment URL to README.

Progress map

Milestone 1 - Tasks 1-8: documents, chunking, embeddings

Milestone 2 - Tasks 9-13: database and tested retrieval

Milestone 3 - Tasks 14-21: working RAG API and citations

Milestone 4 - Tasks 22-23: Next.js interface

Milestone 5 - Tasks 24-30: hybrid retrieval, reranking, conversational queries

Milestone 6 - Tasks 31-35: evaluation and debug dashboard

Milestone 7 - Tasks 36-38: deployment and documentation

Learning rule: Build loading, chunking, embedding, retrieval, context, and citations yourself through Task 18. Add a framework later only if it helps.